

ANÁLISIS Y DISEÑO DE ALGORITMOS — 2026-1

Optimización de Tareas en Servidores

Trabajo Final: Caso de Estudio Algorítmico

Contexto y Relevancia

Eficiencia en la Nube

En sistemas de cómputo modernos, la asignación inteligente de tareas determina la **latencia** y el **costo operativo**.

Este estudio aborda la distribución de n tareas sobre m servidores, buscando el equilibrio entre velocidad de ejecución y balance de carga.

Aplicamos 5 técnicas algorítmicas para comparar **optimalidad vs. rendimiento**.



Definición del Problema



Entradas del Sistema

- **Tareas:** Tiempo, prioridad y dependencias.
- **Servidores:** Capacidad limitada de procesamiento paralelo.
- **Dataset:** Casos desde pequeño (3) hasta grande (12 tareas).



Salida Esperada

- Asignación de cada tarea a un servidor específico.
- Tiempo total de ejecución (Makespan) minimizado.
- Respeto absoluto de restricciones lógicas.

Restricciones del Sistema



Dependencias

Una tarea no puede iniciar hasta que sus predecesoras finalicen.



Prioridades

Las tareas más críticas deben ser atendidas con preferencia en la cola.



Balance

Distribución equitativa para evitar cuellos de botella en servidores.

Demostración Algorítmica

COMPARATIVA: FUERZA BRUTA VS. GREEDY

Fuerza Bruta

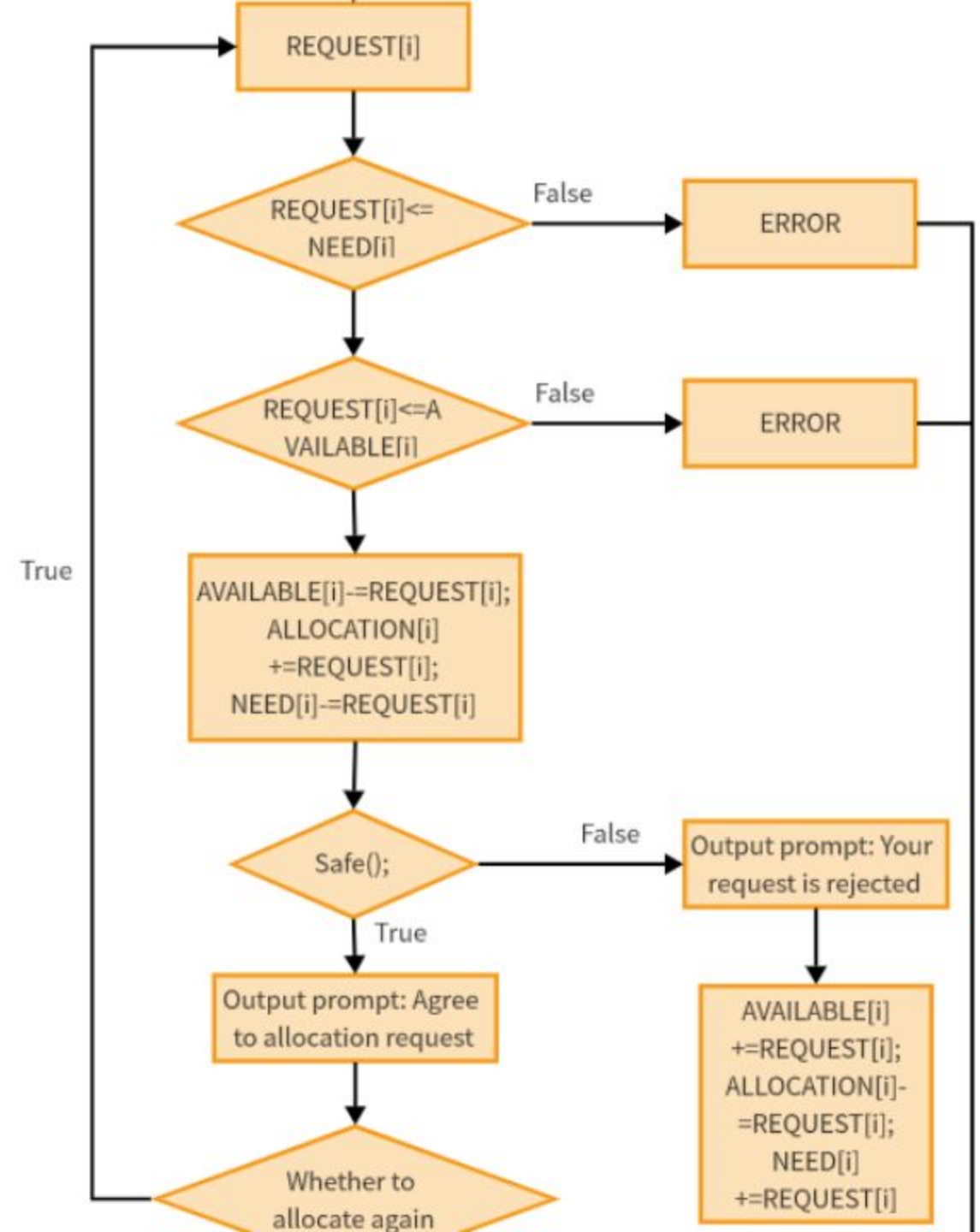
Enfoque Exhaustivo

Explora **todas** las combinaciones posibles de asignación.

Complejidad: m^n

Ventaja: Garantiza siempre la solución óptima absoluta.

Desventaja: Inviabile para $n > 15$ tareas por crecimiento exponencial.



Implementación: Brute Force

```
# Generar TODAS las asignaciones posibles
for asignacion in product(range(num_servidores), repeat=len(tareas)):
    cargas = [0] * num_servidores
    ejecutadas = set()
    # Evaluar si la asignación respeta dependencias
    # y prioridades en cada paso...
    tiempo_total = max(cargas)
    if tiempo_total < mejor_tiempo:
        mejor_tiempo = tiempo_total
```

Utiliza *itertools.product* para construir el espacio de búsqueda completo.

Técnica: Greedy (Voraz)

Heurística de Decisión

En cada paso, el algoritmo toma la **mejor decisión local**:

- Selecciona la tarea disponible de mayor prioridad.
- La asigna al servidor que tenga la menor carga actual.

Complejidad: $O(n^2)$

Ideal para sistemas de tiempo real.

```
oyee name    : ", stdout);          /* no conversion - use fpu
, MAXC, stdin) == NULL) {          /* read all input with fge
ser canceled input)\n", stderr);    /* validate EVERY input */

(buf, "\n\n");                      /* length before either "\
NM) {                               /* validate length */
ror: name too long.\n", stderr);

                                     /* trim trailing '\n' */

e, buf, len + 1);                   /* copy to name */
```


Implementación: Greedy

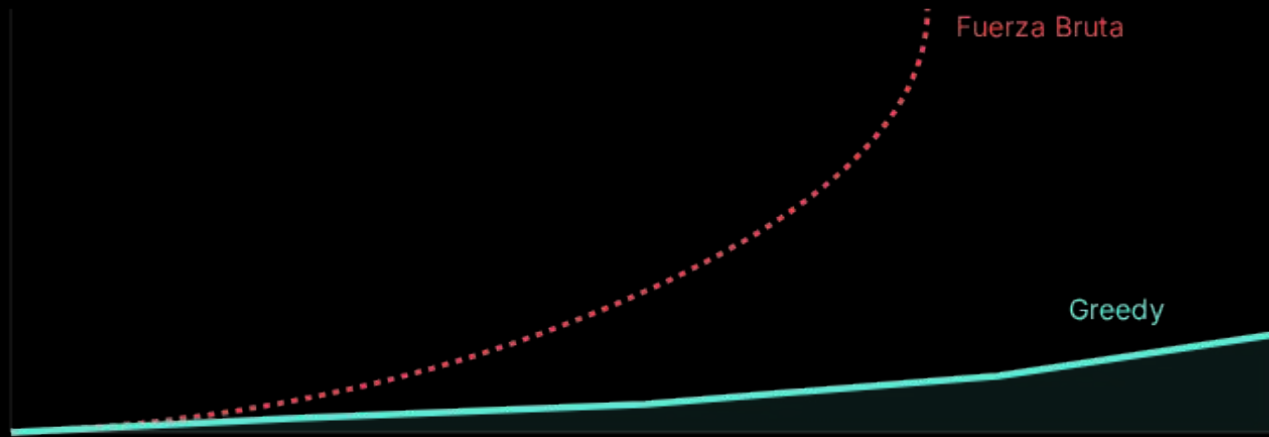
```
while len(ejecutadas) < len(tareas):  
    disponibles = obtener_disponibles(tareas, ejecutadas)  
    disponibles.sort(key=lambda x: x[2]) # Prioridad  
    tarea = disponibles[0]  
  
    # Elegir servidor menos cargado (Decisión Greedy)  
    servidor = cargas.index(min(cargas))  
    cargas[servidor] += tarea[1]  
    ejecutadas.add(tarea[0])
```

Rápido y eficiente, aunque no siempre alcanza el óptimo global.

Comparativa de Paradigmas

Técnica	Complejidad	¿Óptima?	Ventaja Principal
Fuerza Bruta	$O(m^n)$	Si	Garantía de exactitud
Greedy	$O(n^2)$	No siempre	Velocidad extrema
Backtracking	$O(m^n)$ c/poda	Si	Eficiencia con precisión
Divide y Vencerás	$O(n \log n)$	No siempre	Escalabilidad masiva

Análisis de rendimiento



Interpretación

La gráfica ilustra el **punto de quiebre** computacional donde los algoritmos exactos dejan de ser prácticos y las heurísticas se vuelven necesarias.

Conclusiones

-  **Validación de Óptimos:** La Fuerza Bruta es indispensable como *baseline* en problemas pequeños para validar la precisión de heurísticas rápidas.
-  **Poder de la Poda:** El Backtracking demostró ser el equilibrio perfecto entre exactitud y tiempo, reduciendo el espacio de búsqueda significativamente.
-  **Escalabilidad Real:** Para sistemas con miles de tareas, el paradigma Greedy o Divide & Vencerás es el único viable bajo restricciones de tiempo real.